

AI MINI PROJECT REPORT

LOAN APPROVAL PREDICTION SYSTEM

Using Artificial Intelligence, Machine Learning and Bank Loan Approval Prediction

Submitted By
Yash Nandanwar
B.Tech Information Technology
Priyadarshini College of Engineering, Nagpur
Academic Year: 2026–27

AI • MACHINE LEARNING • FINANCIAL TECHNOLOGY

1. Aim

To design and implement an AI/ML-based Loan Approval Prediction System that analyzes an applicant's personal, financial and loan-related information and predicts whether the loan is likely to be Approved or Not Approved.

2. Problem Definition

Loan approval depends on several applicant and financial factors. Manually evaluating all these factors can be time-consuming and inconsistent. The proposed system learns patterns from historical loan application records and predicts the likely decision for a new applicant.

3. Abstract

The Loan Approval Prediction System is an AI/ML mini project developed using Python, Pandas, Scikit-learn and Flask. The system uses applicant information such as income, co-applicant income, loan amount, loan term, credit score, existing loans, savings, education, employment status, marital status, dependents and property area. Categorical values are converted into numerical representations using One-Hot Encoding, while missing values are handled using imputation. A Random Forest Classifier with 200 estimators and a maximum depth of 10 is trained on the prepared dataset. The web dashboard accepts a new applicant profile and displays the predicted loan decision together with approval/rejection probabilities and a simple risk category.

4. Objectives

- To develop a simple AI/ML-based loan approval prediction system.
- To prepare and use a structured loan application dataset.
- To preprocess categorical and numerical applicant features.
- To train a Random Forest classification model.
- To predict whether a loan is likely to be approved or not approved.
- To display prediction confidence through approval and rejection probabilities.
- To provide the prediction through a professional web-based dashboard.

5. Introduction

Loan approval prediction is a binary classification problem. Machine learning can identify patterns in historical applications and apply those patterns to new cases. This project demonstrates dataset preparation, preprocessing, model training, evaluation, web-based prediction and result visualization.

6. Technologies Used

Component	Technology
Programming Language	Python
Web Framework	Flask
Data Processing	Pandas and NumPy
Machine Learning	Scikit-learn
AI Technique Classification	Supervised Machine Learning •
Algorithm Preprocessing	trained preprocessing pipeline
Frontend	HTML, CSS and JavaScript
Development Environment	Visual Studio Code

7. AI / Machine Learning Technique

7.1 Supervised Classification

The model learns from labelled loan records where each row contains input features and a known Loan_Status value. It then predicts the class of an unseen application.

7.2 Random Forest Classifier

Random Forest combines multiple decision trees and is suitable for mixed applicant information after preprocessing. It can model non-linear relationships and provide class probabilities.

7.3 One-Hot Encoding and Imputation

Categorical fields are transformed with One-Hot Encoding. Missing categorical values use the most frequent value and

missing numerical values use the median.

8. Dataset and Features

The included dataset contains 800 records and 15 columns. There are 96 missing cells among model input features; preprocessing handles them automatically.

Feature	Type	Description
Gender	Categorical	Male / Female
Married	Categorical	Yes / No
Dependents	Categorical	0 / 1 / 2 / 3+
Education	Categorical	Graduate / Not Graduate
Self_Employed	Categorical	Yes / No
Property_Area	Categorical	Urban / Semiurban / Rural
Applicant_Income	Numeric	Applicant income Co-
Coapplicant_Income	Numeric	applicant income
Loan_Amount	Numeric	Requested loan amount
Loan_Amount_Term	Numeric	Loan repayment term
Credit_Score	Numeric	Applicant credit score
Existing_Loans	Numeric	Number of existing loans

Feature	Type	Description
Savings	Numeric	Available savings
Loan_Status	Target	0 = Not Approved, 1 = Approved

9. System Requirements

- Computer/laptop; minimum 4 GB RAM recommended.
- Python 3.x and a Python IDE such as Visual Studio Code.
- Modern web browser.
- Packages listed in requirements.txt.

10. Input Parameters

- Gender
- Married
- Dependents
- Education
- Self_Employed
- Property_Area
- Applicant_Income
- Coapplicant_Income
- Loan_Amount
- Loan_Amount_Term
- Credit_Score
- Existing_Loans
- Savings

11. Target Classes

- Approved(1) — higher likelihood of approval.
- Not Approved (0) — higher likelihood of rejection.

The dashboard uses a demonstration risk label: LOW at 70% or more approval probability, MEDIUM from 45% to 69.99%, and HIGH below 45%. This is not a banking/regulatory risk score.

12. Working / Methodology

- Load the loan approval CSV dataset.
- Separate the input features and Loan_Status target.
- Divide categorical and numerical features.
- Apply missing-value imputation.
- Apply One-Hot Encoding to categorical features.
- Split the dataset into training and testing sets using an 80:20 ratio.
- Train the Random Forest Classifier with 200 estimators.
- Evaluate the model on the held-out test data.
- Accept a new applicant profile through the web dashboard.
- Pass the input through the same preprocessing pipeline.
- Predict the loan class and calculate approval/rejection probabilities.
- Display the result and risk category in the dashboard.

13. Algorithm

- Start the Flask application.
- Read the loan application dataset.
- Separate features and target.
- Impute missing values and encode categorical features.
- Split data into training and testing sets.
- Train the Random Forest classifier.
- Read new applicant details from the web form.
- Apply the trained preprocessing pipeline.
- Predict the loan approval class.
- Calculate class probabilities.
- Determine the demonstration risk category.
- Display the prediction and probabilities.
- Stop / wait for the next prediction request.

14. Program

Core model configuration:

```
from flask import Flask, jsonify, render_template, request
import pandas as pd
from sklearn.compose import ColumnTransformer
from sklearn.ensemble import RandomForestClassifier
from sklearn.impute import SimpleImputer
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder

classifier = RandomForestClassifier(
    n_estimators=200,
    max_depth=10,
    random_state=42,
    class_weight="balanced",
    n_jobs=-1
)
```

Prediction and probability calculation:

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20,
    random_state=42, stratify=y
)

model.fit(X_train, y_train)

prediction = model.predict(X_new)[0]
probabilities = model.predict_proba(X_new)[0]

approval_probability = probabilities[1] * 100
rejection_probability = probabilities[0] * 100
```

The complete implementation is provided in app.py, with the web interface in templates/index.html, static/style.css and static/script.js.

15. Sample Output

Sample applicant profile and the corresponding prediction from the included dataset/model:

Input	Value
Gender	Male
Married	Yes
Dependents	0
Education	Graduate
Self Employed	No
Property Area	Semiurban
Applicant Income	55,000
Co-applicant Income	25,000
Loan Amount	180
Loan Term	360
Credit Score	720
Existing Loans	1
Savings	350,000

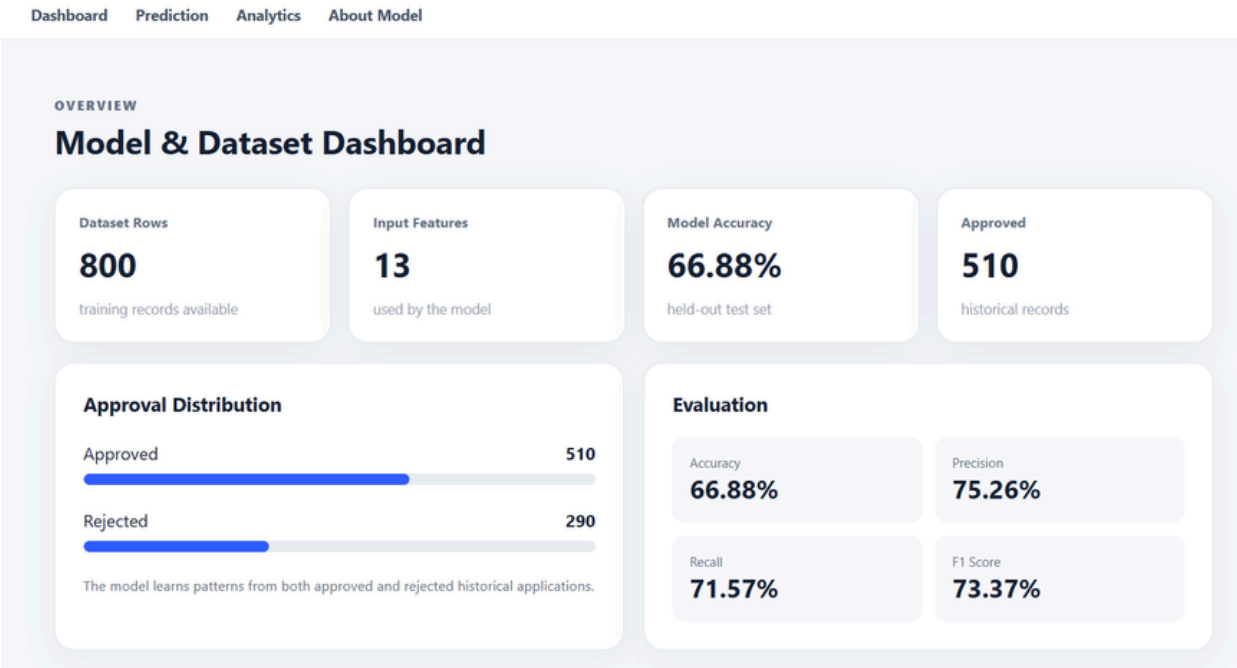
Sample prediction

Result	Value
Prediction	LOAN APPROVED
Approval probability	79.44%
Rejection probability	20.56%
Risk category	LOW

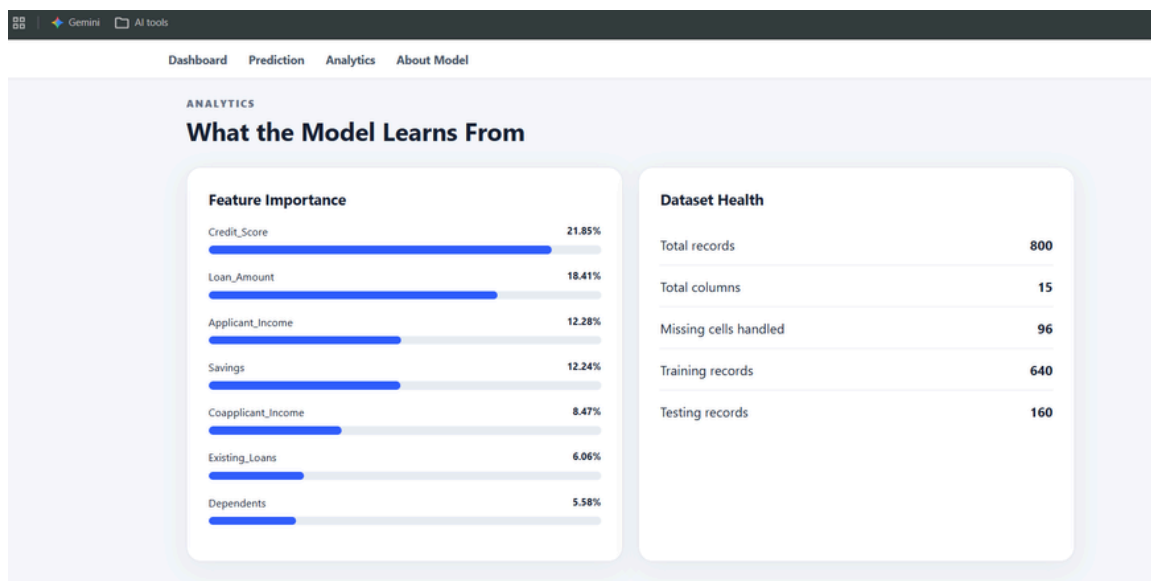
This is a machine-learning demonstration and not an actual lending decision.

16. Application Dashboard Output

The dashboard presents dataset statistics, loan-status distribution, model evaluation, grouped feature importance and a prediction form.



17. Model Evaluation



18. Advantages

- Simple webinterface.
- Actual supervised ML model.
- Categorical and numerical features.
- Automatic missing-value handling.
- Approval/rejection probabilities.
- Evaluation metrics and feature importance.
- Complete source code and dataset included.

19. Limitations

- Dataset issynthetic/prepared for academic demonstration.
- Results depend on training-data quality.
- Not all real-world lending factors are included.
- Model probability is not a guaranteed chance of loan approval.
- Not intended for real-world financial decisions.

20. Future Scope

- Use larger official datasets.
- Add debt-to-income ratio and detailed credit history.
- Compare multiple ML algorithms.
- Add explainable AI.
- Add model versioning and periodic retraining.
- Cloud deployment.
- Secure authentication and database storage.
- Fairness/bias analysis.

21. Screenshots / Output

[Dashboard](#) [Prediction](#) [Analytics](#) [About Model](#)

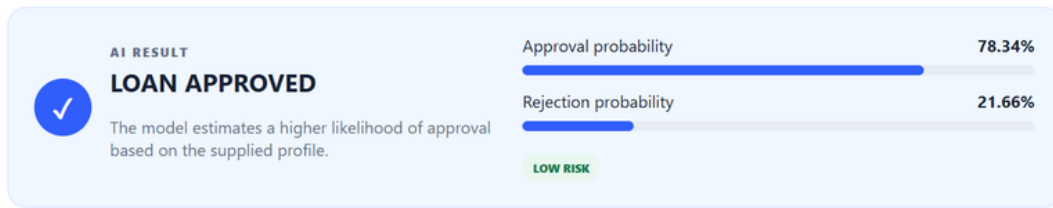
Analyze a New Loan Application

Enter applicant information and let the trained ML model estimate the approval outcome.

Gender Male	Married Yes	Dependents 0
Education Graduate	Self Employed No	Property Area Urban
Applicant Income (₹ / month) 55000	Co-applicant Income (₹ / month) 25000	Loan Amount (₹ thousand) 180
Loan Term (months) 360	Credit Score 720	Existing Loans 1
Savings (₹) 350000		

Predict Loan Approval

22. Final Output Evidence



Project files included in the original project: app.py, loan_approval_data.csv, templates/index.html, static/style.css, static/script.js, requirements.txt, README.md, and dataset_info.py.

Run commands

```
pip install -r requirements.txt
python app.py
```

Then open <http://127.0.0.1:5000> in a browser.